ANN & Deep Learning

Comprehensive Study Notes

CNNs · Dense Prediction · RNNs · Tokenization · Word Embeddings · Attention & Transformers

Table of Contents

1. Convolutional Neural Networks (CNNs)

Convolutional Neural Networks (CNNs) are a class of deep neural networks specifically designed to process data with a **grid-like topology** — such as images (2D grids of pixels), audio (1D time series), or video (3D). Introduced by Yann LeCun (1989, LeNet), they use the **convolution operation** instead of general matrix multiplication in at least one layer, enabling automatic feature extraction through learnable filters.

The key insight is **weight sharing**: the same filter (a small matrix of weights) is applied at every position of the input, drastically reducing parameters compared to fully connected networks and encoding the assumption that features (edges, textures) can appear anywhere in an image.

1.1 Why Not Fully Connected for Images?

Worked Example

A 224x224 RGB image has 224 x 224 x 3 = 150,528 input values.

A fully connected hidden layer with 1024 neurons needs:

150,528 x 1024 = 154,140,672 weights ← just ONE layer!

A CNN conv layer with 64 filters of size 3x3x3:

64 x (3x3x3 + 1) = 64 x 28 = 1,792 parameters ← 85,000x fewer

Weight sharing also provides translation equivariance:

A filter detecting a vertical edge works anywhere in the image.

1.2 The Convolution Operation

A **filter** (also called kernel) is a small 2D or 3D matrix of learnable weights. It slides across the input feature map, computing the **dot product** between filter weights and the local input patch at each position. This produces one value in the **output feature map**.

Output(i,j) = sum over (m,n): Input(i+m, j+n) x Filter(m,n) + bias

For 3D input (C channels): Output(i,j) = sum over (c,m,n): Input(c, i+m, j+n) x W(c,m,n) + b

Worked Example

5x5 input, 3x3 filter, stride=1, no padding:

Input patch at (0,0): Filter:

1 2 3 1 0 -1

4 5 6 x 2 0 -2

7 8 9 1 0 -1

Output(0,0) = 1*1 + 2*0 + 3*(-1) + 4*2 + 5*0 + 6*(-2) + 7*1 + 8*0 + 9*(-1)

```
= 1 + 0 - 3 + 8 + 0 - 12 + 7 + 0 - 9 = -8

This is a Sobel filter for horizontal edge detection!
```

1.3 Key Spatial Hyperparameters

Padding

Without padding, feature maps shrink with every convolution. **Same (zero) padding** adds zeros around the border so output matches input spatial size. **Valid padding** means no padding; output shrinks.

```
With same padding, stride=1: Output size = Input size
With valid padding: Output size = floor((Input - Filter + 1) / stride)
```

Stride

Stride controls how many pixels the filter moves at each step. Stride=1 → dense, overlapping. Stride=2 → halves spatial dimensions (like pooling). Large strides are sometimes used instead of pooling.

```
Output spatial dim = floor((Input - Filter + 2*Padding) / Stride) + 1
```

Worked Example

```
Input: 32x32, Filter: 5x5, Padding: 2 (same), Stride: 1
Output = floor((32 - 5 + 2*2) / 1) + 1 = floor(31/1) + 1 = 32x32

Input: 32x32, Filter: 5x5, Padding: 0, Stride: 2
Output = floor((32 - 5 + 0) / 2) + 1 = floor(27/2) + 1 = 14x14
```

1.4 Activation Functions

After convolution, a non-linear activation function introduces non-linearity — without it, stacking layers would still only learn linear transformations.

Activation	Formula	Range	Key Property
ReLU	max(0, x)	[0, inf)	Fast, sparse, vanishing gradient free for x>0
Leaky ReLU	x if x>0 else 0.01x	(-inf, inf)	Fixes dead neuron problem
ELU	x if x>0 else a(e^x - 1)	(-a, inf)	Smooth negative, mean closer to 0
GELU	x * Phi(x) (Gaussian CDF)	(-inf, inf)	Used in Transformers, BERT, GPT
Sigmoid	1 / (1 + e^(-x))	(0, 1)	Squashes to probability; vanishing grad
Tanh	(e^x - e^(-x)) / (e^x + e^(-x))	(-1, 1)	Zero-centred; still vanishing grad
Softmax	e^(xi) / sum(e^(xj))	(0,1), sums to 1	Multi-class output probabilities

1.5 Pooling Layers

Pooling reduces spatial dimensions (downsampling), making features more compact and providing **spatial invariance** to small translations. It has no learnable parameters.

Type	Operation	Property
Max Pooling	Take maximum value in each window	Preserves strongest activation; most common
Average Pooling	Take mean value in each window	Smoother; used in GAP (Global Avg Pool)
Global Avg Pool	Average over entire feature map -> scalar	Replaces FC layers; reduces overfitting
Global Max Pool	Max over entire feature map -> scalar	Keeps most prominent feature per map

Worked Example

```
2x2 Max Pooling with stride=2 on 4x4 feature map:

Input:          Output:
1 3 2 4         3 4
5 6 7 8   ->    6 9
1 2 3 4         3 9
9 1 8 5         9 8

Spatial size: 4x4 -> 2x2 (halved in each dimension)
```

1.6 Batch Normalization

Batch Normalization (Ioffe & Szegedy, 2015) normalises activations within a mini-batch, dramatically accelerating training, enabling higher learning rates, and acting as a regulariser.

```
mu_B = (1/m) * sum(xi) [batch mean]
sigma_B^2 = (1/m) * sum((xi - mu_B)^2) [batch variance]
x_hat_i = (xi - mu_B) / sqrt(sigma_B^2 + epsilon) [normalize]
y_i = gamma * x_hat_i + beta [scale and shift; gamma,beta are learnable]
```

Where BatchNorm is placed

Original paper: Conv -> BN -> Activation. Modern practice often: Conv -> Activation -> BN. In Transformers: Layer Normalization is used instead (normalises over feature dimension, not batch dimension — works better for variable-length sequences).

1.7 Dropout Regularization

During training, each neuron is randomly set to zero with probability p (typically 0.5 for FC layers, 0.1-0.2 for conv layers). At test time, all neurons are used but outputs are scaled by (1-p). This prevents co-adaptation of neurons and acts as an ensemble of 2^n networks.

```
During training: h = mask * activation, where mask ~ Bernoulli(1-p)
During inference: h = (1-p) * activation [weight scaling / inverted dropout]
```

2. CNNs for Image Classification

Image classification assigns one label from a fixed set of classes to an entire image. CNNs have achieved superhuman performance on benchmarks like ImageNet (1000 classes, 1.28M images). The general pipeline: Input Image → Stack of Conv+Pool blocks → Flatten → FC layers → Softmax → Class.

2.1 The Classification Pipeline

```
Input: H x W x C image (e.g. 224x224x3 RGB)
-> Conv Block 1: learn low-level features (edges, colours, gradients)
-> Conv Block 2: learn mid-level features (corners, curves, textures)
-> Conv Block 3+: learn high-level features (object parts, faces, wheels)
-> Global Average Pooling or Flatten
-> Fully Connected layer(s)
-> Softmax: output probability distribution over K classes
Loss: Cross-Entropy = -sum_k( y_k * log(p_k) )
```

2.2 Landmark Architectures

LeNet-5 (LeCun, 1998) — The Pioneer

- Input: 32x32 grayscale (handwritten digits, MNIST).
- Architecture: Conv(6@5x5) -> AvgPool(2x2) -> Conv(16@5x5) -> AvgPool(2x2) -> FC(120) -> FC(84) -> Softmax(10).
- First demonstration that CNNs outperform other methods on vision tasks.
- Total parameters: ~60,000 — tiny by modern standards.

AlexNet (Krizhevsky et al., 2012) — The Deep Learning Revolution

- Won ImageNet 2012 with top-5 error of 15.3% vs 26.2% for second place — a 10-point gap.
- First use of ReLU (instead of tanh/sigmoid) in deep networks — enabling depth without vanishing gradients.
- First use of GPU training (2× NVIDIA GTX 580), dropout, data augmentation.
- Architecture: 5 conv layers + 3 FC layers, ~60M parameters.
- Input: 227x227x3. Max pooling, local response normalisation.

Worked Example
AlexNet Layer-by-Layer:
Input: 227x227x3
Conv1: 96 filters, 11x11, stride=4 -> 55x55x96 -> MaxPool -> 27x27x96
Conv2: 256 filters, 5x5, pad=2 -> 27x27x256 -> MaxPool -> 13x13x256
Conv3: 384 filters, 3x3, pad=1 -> 13x13x384
Conv4: 384 filters, 3x3, pad=1 -> 13x13x384

```
Conv5: 256 filters, 3x3, pad=1 -> 13x13x256 -> MaxPool -> 6x6x256
FC6: 4096 neurons (dropout 0.5)
FC7: 4096 neurons (dropout 0.5)
FC8: 1000 neurons -> Softmax
```

VGGNet (Simonyan & Zisserman, 2014) — Deep and Simple

- Key insight: replace large filters with **stacks of small 3x3 filters**.
- Two 3x3 conv layers have the same receptive field as one 5x5 but fewer parameters and more non-linearity.
- Three 3x3 = one 7x7 in receptive field; params: $3 \times 9C^2$ vs $49C^2$ (cheaper and deeper).
- VGG-16: 16 weight layers; VGG-19: 19 layers. ~138M parameters. Top-5 error: 7.3%.
- Simple, uniform architecture — very popular as a feature extractor backbone.

GoogLeNet / Inception (Szegedy et al., 2014) — Inception Modules

- Won ImageNet 2014 with top-5 error of 6.67%. Only 5M parameters — 12x fewer than AlexNet.
- **Inception module**: apply 1x1, 3x3, and 5x5 convolutions and 3x3 max pooling in parallel; concatenate outputs.
- **1x1 convolutions** (bottleneck): reduce channel depth before expensive 3x3/5x5 convolutions, cutting computation.
- Uses Global Average Pooling instead of large FC layers.
- Auxiliary classifiers during training to combat vanishing gradients in deep networks.

```
Inception bottleneck example:
Input: H x W x 256 channels
1x1 conv (32 filters) -> H x W x 32 [reduce channels]
3x3 conv (128 filters) -> H x W x 128 [main conv, cheaper now]
```

ResNet (He et al., 2015) — Residual Learning

- Won ImageNet 2015 with top-5 error 3.57% — better than human (~5%).
- Solved the **degradation problem**: deeper networks were performing worse, not better.
- **Residual (skip) connections**: output = F(x) + x — learn residual F(x) = desired_output - x.
- If optimal mapping is identity, F(x) is driven to zero — easier to learn than the full mapping.
- Enabled training of 152-layer networks (ResNet-152) and later 1000+ layer variants.
- The shortcut connection is added element-wise (1x1 conv if dimensions differ).

```
Residual block: y = F(x, {Wi}) + x
F(x) = W2 * ReLU(BN(W1 * ReLU(BN(x)))) [2-layer residual]
If dims differ: y = F(x, {Wi}) + Ws*x [projection shortcut]
```

Worked Example

```
Why residual connections help:
Without skip: gradients must flow through every layer — vanish by layer 50+
```

```
With skip: gradient = dL/dy * 1 (from identity path) always provides
strong gradient signal to early layers
Gradient: dL/dx = dL/dy * (dF/dx + 1) <- the '+1' ensures flow
```

EfficientNet (Tan & Le, 2019) — Compound Scaling

- Systematically scales width, depth, and resolution together using a compound coefficient phi.
- Found by neural architecture search (NAS); achieves state-of-the-art accuracy with fewer FLOPs.
- EfficientNet-B7: ImageNet top-1 accuracy 84.3% with 66M parameters.

Vision Transformer (ViT, Dosovitskiy et al., 2020)

- Applies the Transformer architecture (designed for NLP) directly to image patches.
- Image is divided into 16x16 patches, each flattened and projected to an embedding.
- Achieves superior performance with large datasets; less inductive bias than CNNs.

2.3 Training CNNs: Loss, Optimisation, Regularisation

Cross-Entropy Loss

```
L = -(1/N) * sum_i [ sum_k ( y_ik * log(p_ik) ) ]
For binary: L = -[ y*log(p) + (1-y)*log(1-p) ]
```

Optimisers

Optimiser	Update Rule	Key Property
SGD	w = w - lr * grad(L)	Simple; needs careful lr tuning
Momentum	v = beta*v - lr*grad; w = w + v	Dampens oscillations; faster convergence
Adam	Adaptive lr per param; 1st and 2nd moment estimates;	Default choice; robust to lr; fast
AdaGrad	lr / sqrt(sum of squared grads)	Good for sparse gradients (NLP)
RMSprop	lr / sqrt(EMA of squared grads)	Fixes AdaGrad's lr decay; good for RNNs
AdamW	Adam + decoupled weight decay	Better generalisation than Adam

Data Augmentation

- **Geometric:** random crop, horizontal flip, rotation (±15°), zoom, translation.
- **Colour jitter:** brightness, contrast, saturation, hue perturbation.
- **CutOut / Random Erasing:** mask random rectangular patches to force global feature learning.
- **Mixup:** linearly interpolate two training images and their labels.
- **CutMix:** cut and paste patches between images; mix labels proportionally.
- **AutoAugment / RandAugment:** learned or random policies of augmentation sequences.

3. Dense Prediction with CNNs

Dense prediction (also called pixel-wise prediction or structured prediction) assigns a label or value to **every pixel** of the input image, rather than producing a single label for the whole image. Key tasks include semantic segmentation, depth estimation, optical flow, surface normal prediction, and saliency detection. The core challenge is maintaining spatial resolution lost by pooling while retaining rich semantic features.

3.1 Fully Convolutional Networks (FCN)

Long et al. (2015) showed that **any CNN for classification can be converted to an FCN** for dense prediction by replacing the fully connected layers with 1x1 convolutions. This allows the network to accept any input size and output a spatial prediction map.

- Replace FC layers with equivalent 1x1 convolutions → preserves spatial information.
- Use **transposed convolution** (deconvolution) to upsample feature maps back to full resolution.
- Output a C-channel heatmap (C = number of classes); argmax gives predicted class per pixel.
- Trained end-to-end with pixel-wise cross-entropy loss.

```
Pixel-wise loss: L = -(1/HW) * sum_(i,j) [ sum_k y_ijk * log(p_ijk) ]
```

3.2 Encoder-Decoder Architecture

A common pattern for dense prediction is the encoder-decoder (hourglass) architecture. The **encoder** progressively reduces spatial resolution while increasing semantic depth. The **decoder** recovers spatial resolution. Skip connections transfer fine spatial details from encoder to decoder to preserve boundary information.

```
Encoder: 224x224 -> 112x112 -> 56x56 -> 28x28 -> 14x14 -> 7x7
Decoder: 7x7 -> 14x14 -> 28x28 -> 56x56 -> 112x112 -> 224x224
```

Upsampling Methods

Method	Description	Learnable?
Bilinear Interpolation	Fixed; smoothly interpolates between known values	No
Nearest Neighbour	Repeat nearest pixel; blocky	No
Transposed Conv	Learnable upsampling; 'fractional stride' convolution	Yes — can introduce artefacts
Sub-pixel Conv (ESPCN)	Rearrange channels into spatial: H x W x C*r^2 -> Hr x Wr x C	Yes
Unpooling	Restore max-pool using stored indices (SegNet)	No

3.3 U-Net in Depth

Originally designed for biomedical segmentation (Ronneberger 2015), U-Net is now the dominant architecture for dense prediction with limited data. Its defining feature is **symmetric skip connections** that concatenate (not add) encoder feature maps into decoder.

Worked Example

```
U-Net Architecture (input 572x572):
```

```
Encoder path (contracting):
Block1: 2x [Conv3x3 -> ReLU] -> 570x570x64 -> MaxPool2x2 -> 284x284x64
Block2: 2x [Conv3x3 -> ReLU] -> 280x280x128 -> MaxPool2x2 -> 140x140x128
Block3: 2x [Conv3x3 -> ReLU] -> 136x136x256 -> MaxPool2x2 -> 68x68x256
Block4: 2x [Conv3x3 -> ReLU] -> 64x64x512 -> MaxPool2x2 -> 32x32x512
Bottleneck: 2x [Conv3x3->ReLU] -> 28x28x1024
Decoder path (expansive):
Upsample -> concat skip from Block4 -> 2x Conv -> ...
Final 1x1 conv -> 388x388 x num_classes

Skip connections preserve: fine edges, texture details, exact boundaries
```

3.4 Dilated (Atrous) Convolutions

Dilated convolutions insert zeros between filter weights, expanding the **receptive field without reducing resolution** and without extra parameters. This is crucial for dense prediction where we need both large context and fine spatial detail.

```
Dilated conv output: y(i) = sum_k x(i + r*k) * w(k)
r = dilation rate; r=1 is standard conv; r=2 skips one pixel between samples
Receptive field with dilation r, filter K: K + (K-1)*(r-1) pixels
```

Worked Example

```
3x3 filter with different dilation rates:
r=1: covers 3x3 = 9 pixels (standard)
r=2: covers 5x5 = 25 pixels (samples 9 of them)
r=4: covers 9x9 = 81 pixels (samples 9 of them)
r=8: covers 17x17 = 289 pixels (samples 9 of them)

Atrous Spatial Pyramid Pooling (ASPP) in DeepLab:
Apply dilated conv with r=6, 12, 18 in parallel -> multi-scale context
```

3.5 DeepLab Family

- **DeepLab v1 (2015):** Dense CRF post-processing on CNN output to sharpen boundaries.
- **DeepLab v2 (2016):** ASPP for multi-scale features; ResNet backbone.
- **DeepLab v3 (2017):** Improved ASPP with image-level global pooling; removes CRF.
- **DeepLab v3+ (2018):** Adds decoder module for sharper object boundaries; Xception backbone; achieves 89.0 mIoU on PASCAL VOC 2012.

3.6 Instance vs Semantic Segmentation

Task	Output	Distinguishes Instances	Key Architecture
Semantic Segmentation	Per-pixel class label	No	FCN, U-Net, DeepLab
Instance Segmentation	Per-pixel + per-instance	Yes	Mask R-CNN, SOLO
Panoptic Segmentation	Semantic + instance unified	Yes	Panoptic FPN, Mask2Former
Depth Estimation	Per-pixel depth value	N/A	BerHu loss; encoder-decoder

3.7 Evaluation: mIoU

```
IoU (per class) = TP / (TP + FP + FN)
mIoU = (1/C) * sum_c IoU_c [mean over all C classes]
Pixel Accuracy = (sum_i TP_i) / (total pixels)
```

Worked Example

```
Class 'car': 1000 pixels predicted as car
TP = 800 (correctly labelled car)
FP = 200 (non-car labelled as car)
FN = 150 (car pixels labelled as non-car)
IoU = 800 / (800 + 200 + 150) = 800 / 1150 = 0.696

mIoU across 5 classes: (0.70 + 0.85 + 0.63 + 0.92 + 0.78) / 5 = 0.776
```

4. Recurrent Neural Networks (RNNs)

Recurrent Neural Networks (RNNs) are designed to process **sequential data** — text, speech, time series, video — by maintaining a **hidden state** that acts as memory of past inputs. Unlike feedforward networks which process fixed-size inputs independently, RNNs share parameters across time steps and can handle variable-length sequences.

4.1 Vanilla RNN

At each time step t, the RNN takes input x_t and the previous hidden state h_{t-1}, combines them with weight matrices, and produces a new hidden state h_t and optionally an output y_t.

```
h_t = tanh(W_hh * h_{t-1} + W_xh * x_t + b_h)
y_t = W_hy * h_t + b_y
Parameters: W_hh (hidden-to-hidden), W_xh (input-to-hidden), W_hy (hidden-to-output)
All time steps SHARE the same weight matrices -> weight sharing across time
```

Worked Example

```
Character-level language model: input 'hell', predict 'ello'

Vocabulary: {h:0, e:1, l:2, o:3} (one-hot encoding)
t=1: x_1=[1,0,0,0] (h) -> h_1 = tanh(W_xh*x_1 + W_hh*h_0) -> y_1 predict 'e'
t=2: x_2=[0,1,0,0] (e) -> h_2 = tanh(W_xh*x_2 + W_hh*h_1) -> y_2 predict 'l'
t=3: x_3=[0,0,1,0] (l) -> h_3 = tanh(W_xh*x_3 + W_hh*h_2) -> y_3 predict 'l'
t=4: x_4=[0,0,1,0] (l) -> h_4 = tanh(W_xh*x_4 + W_hh*h_3) -> y_4 predict 'o'
```

4.2 Backpropagation Through Time (BPTT)

Training RNNs requires computing gradients through time steps — unrolling the network across T steps and applying standard backprop. This leads to the **vanishing/exploding gradient problem**.

```
dL/dW = sum_{t=1}^{T} dL_t/dW
Gradient at step k: dL_T/dh_k = dL_T/dh_T * PRODUCT_{t=k+1}^{T} (W_hh^T * diag(f'(h_t)))
If |W_hh| < 1: gradient -> 0 as T-k grows [VANISHING]
If |W_hh| > 1: gradient -> inf as T-k grows [EXPLODING]
```

- **Gradient clipping**: cap gradient norm to threshold (e.g. 1.0) to prevent explosion.
- **Truncated BPTT**: only backprop through last k steps instead of all T.
- **Exploding**: easy to fix with gradient clipping.
- **Vanishing**: fundamentally difficult — solved by LSTM/GRU architectures.

4.3 Long Short-Term Memory (LSTM)

Invented by Hochreiter & Schmidhuber (1997), LSTMs solve the vanishing gradient problem with a **cell state** C_t — a memory conveyor belt — protected by learnable **gates** that control what to remember, forget, and

output. The cell state can maintain information across thousands of time steps.

LSTM Gate Equations

```
Forget gate: f_t = sigmoid(W_f * [h_{t-1}, x_t] + b_f) -> what to ERASE from cell
Input gate: i_t = sigmoid(W_i * [h_{t-1}, x_t] + b_i) -> what to WRITE to cell
Candidate: C~_t = tanh(W_C * [h_{t-1}, x_t] + b_C) -> new candidate values
Cell update: C_t = f_t * C_{t-1} + i_t * C~_t -> UPDATE cell state
Output gate: o_t = sigmoid(W_o * [h_{t-1}, x_t] + b_o) -> what to OUTPUT
Hidden state: h_t = o_t * tanh(C_t) -> output + next hidden
```

Worked Example

```
LSTM processing: 'The cat, which ate the mouse, was ...'

At 'cat': Cell stores [subject=cat, number=singular]
Forget gate: f ~= 1 (keep subject information)
Input gate: i ~= 0 (no new subject to write)
...
At 'was': Cell still contains [subject=cat] from many steps ago
Output: 'was' (singular) not 'were' (plural) -- long-range dependency captured!

Without LSTM (vanilla RNN): gradient vanishes over the long gap -> loses 'cat'
```

4.4 Gated Recurrent Unit (GRU)

GRU (Cho et al., 2014) simplifies LSTM by merging the cell state and hidden state, and using only 2 gates instead of 3. Often matches LSTM performance with fewer parameters.

```
Reset gate: r_t = sigmoid(W_r * [h_{t-1}, x_t])
Update gate: z_t = sigmoid(W_z * [h_{t-1}, x_t])
New memory: n_t = tanh(W_n * [r_t * h_{t-1}, x_t])
Output: h_t = (1 - z_t) * h_{t-1} + z_t * n_t
```

Property	Vanilla RNN	LSTM	GRU
Gates	0	3 (forget, input, output)	2 (reset, update)
States	h only	h and C (cell state)	h only
Parameters/unit	Least	Most (4x RNN)	More than RNN (3x)
Long-range deps.	Poor (vanishing grad)	Excellent	Good
Training speed	Fastest	Slowest	Faster than LSTM
Typical use	Short sequences	Long sequences, language	Similar to LSTM

4.5 Bidirectional RNNs

A standard RNN only sees past context. A **Bidirectional RNN** runs one RNN forward (left-to-right) and one backward (right-to-left), then concatenates both hidden states. This gives each position access to both past and future context — crucial for tasks like NER, sentiment analysis, and machine translation encoding.

```
h_t_forward = RNN_forward(x_1, x_2, ..., x_t)
h_t_backward = RNN_backward(x_T, x_{T-1}, ..., x_t)
h_t = [ h_t_forward ; h_t_backward ] (concatenation, dims doubled)
```

4.6 Sequence-to-Sequence (Seq2Seq) with Attention

Seq2Seq (Sutskever et al., 2014) uses an encoder RNN to compress the input sequence into a fixed-length context vector c, then a decoder RNN to generate the output sequence from c. The bottleneck of a single vector limits performance on long sequences — solved by attention (Section 6).

```
Encoder: h_T = LSTM(x_1, x_2, ..., x_T) [final hidden state = context c]
Decoder: s_t = LSTM(y_{t-1}, s_{t-1}, c) [generate output step by step]
Output: p(y_t) = softmax(W * s_t)
```

Worked Example

```
Machine Translation: 'I love Paris' -> 'J aime Paris'

Encoder reads: x_1='I', x_2='love', x_3='Paris'
Context vector: c = h_3 (compressed representation of whole sentence)

Decoder generates:
s_0 = c
y_1 = argmax(softmax(W*s_0)) = 'J'
s_1 = LSTM('J', s_0, c)
y_2 = argmax(softmax(W*s_1)) = 'aime'
s_2 = LSTM('aime', s_1, c)
y_3 = argmax(softmax(W*s_2)) = 'Paris'
y_4 = '' (end of sequence token)
```

5. Tokenization and Word Embeddings

Before feeding text into any neural network, it must be converted from raw strings into numerical representations. This involves two steps: **tokenization** (splitting text into discrete units called tokens) and **embedding** (mapping tokens to dense numeric vectors in a continuous space where semantically similar words are close together).

5.1 Tokenization

Character-Level Tokenization

- Split text into individual characters: 'Hello' -> ['H','e','l','l','o']
- Vocabulary size: ~100 (ASCII/Unicode subset). No out-of-vocabulary (OOV) problem.
- Cons: very long sequences; no morphological structure captured.
- Used in character-level language models, certain spell-checking systems.

Word-Level Tokenization

- Split by whitespace/punctuation: 'Hello world' -> ['Hello', 'world']
- Vocabulary: 50,000–500,000 words (common: 30k-100k).
- Pro: tokens align with human understanding. Con: OOV words (typos, new words) get UNK token.
- Inflections create vocabulary explosion: 'run', 'runs', 'ran', 'running' are separate tokens.

Subword Tokenization (Modern Standard)

Balances character and word tokenization. Frequent words remain whole; rare words split into subwords. No OOV problem — any word can be decomposed into known subwords.

Algorithm	Approach	Used In
Byte Pair Encoding (BPE)	Iteratively merge most frequent character pair until vocab size reached	GPT, GPT-2/3/4, RoBERTa
WordPiece	Like BPE but merges maximise LM likelihood, not pair frequency	BERT, DistilBERT
SentencePiece	Treats input as raw bytes; language-agnostic, no pre-tokenization	T5, ALBERT, mT5
Unigram LM	Starts large, prunes tokens that least decrease likelihood	XLNet

Worked Example

```
BPE Tokenization example:
Input: 'unbelievable'
Vocabulary contains: 'un', 'believ', 'able'
Tokenized: ['un', 'believ', 'able'] (3 tokens, not 13 chars, not 1 word)

'unhappiness' -> ['un', 'happi', 'ness']
'ChatGPT' -> ['Chat', 'G', 'PT'] (GPT-4 tokenizer)
' hello' -> [' hello'] (spaces included in token in GPT tokenizer)

Typical vocabulary sizes:
```

```
GPT-2/3/4: 50,257 tokens
BERT: 30,522 tokens (WordPiece)
T5: 32,000 tokens (SentencePiece)
```

5.2 One-Hot Encoding (Baseline)

Each word in a vocabulary of size V is represented as a V-dimensional binary vector with a 1 at its index and 0s elsewhere. Extremely sparse and high-dimensional; no semantic similarity (all words are equidistant in cosine similarity).

```
V = 10,000 words: vector for 'cat' = [0,0,...,1,...,0] (9,999 zeros, one 1)
dot('cat', 'dog') = dot('cat', 'airplane') = 0 <- no similarity captured
```

5.3 Word2Vec

Mikolov et al. (2013) introduced Word2Vec — shallow neural networks that learn dense d-dimensional word embeddings (typically d=100-300) by predicting context from words or words from context. The core insight: words with similar contexts have similar meanings.

CBOW (Continuous Bag of Words)

Predict the **centre word** from surrounding context words.

```
P(w_t | w_{t-k}, ..., w_{t-1}, w_{t+1}, ..., w_{t+k}) = softmax(W' *
mean_embed(context))
```

Skip-Gram

Predict the **context words** from the centre word. Better for rare words.

```
P(w_c | w_t) = softmax(v_c^T * v_t) for each context word w_c
```

Negative Sampling

Computing full softmax over 30k-100k vocabulary is expensive. Negative sampling approximates it: for each positive (word, context) pair, sample k negative context words and train a binary classifier.

```
Objective: log sigma(v_c^T v_t) + sum_{i=1}^{k} E[log sigma(-v_ni^T v_t)]
```

Worked Example

```
Word2Vec geometric properties:

v('king') - v('man') + v('woman') ~= v('queen') [analogy arithmetic]
v('Paris') - v('France') + v('Italy') ~= v('Rome') [country-capital]
v('walking') - v('walked') ~= v('swimming') - v('swam') [verb tense]

cosine_similarity('cat', 'dog') = 0.82 (similar)
cosine_similarity('cat', 'democracy') = 0.12 (unrelated)
```

5.4 GloVe (Global Vectors for Word Representation)

Pennington et al. (2014). Instead of window-based prediction, GloVe uses global word-word co-occurrence statistics across the entire corpus, directly factorising the co-occurrence matrix.

```
Objective: J = sum_{i,j} f(X_ij) * (v_i^T * v_j + b_i + b_j - log X_ij)^2
X_ij = co-occurrence count of words i and j in a context window
f(x) = (x/x_max)^alpha if x < x_max, else 1 [weighting function]
```

- Captures global corpus statistics more directly than Word2Vec.
- Common d=100, 200, 300 with 6B, 42B, 840B token corpora.
- Similar geometric properties to Word2Vec (analogies work).

5.5 FastText

Bojanowski et al. (2017). Extends Word2Vec by representing each word as a **bag of character n-grams**. The word vector is the sum of its n-gram vectors.

```
v('apple') = v('') + v('')
```

- Handles OOV words: 'unhappiest' can be represented even if never seen in training.
- Better for morphologically rich languages (Turkish, Finnish, Arabic).
- Captures subword semantics: 'unbelievable' shares subword vectors with 'unimaginable'.

5.6 Contextual Embeddings (ELMo, BERT)

Word2Vec, GloVe, and FastText produce **static embeddings** — one fixed vector per word regardless of context. But 'bank' (financial institution) vs 'bank' (river bank) should have different representations. **Contextual embeddings** solve this.

Model	Approach	Representation
ELMo (2018)	BiLSTM trained on language modelling; concatenate all layers	Context-dependent, but RNN-based
BERT (2018)	Transformer encoder; masked language model + NSP pretraining	Deep bidirectional context
GPT (2018)	Transformer decoder; left-to-right language model	Unidirectional; better for generation
RoBERTa	BERT with better training: more data, no NSP, larger batches	Improved BERT
ALBERT	BERT with parameter sharing + factorised embedding	Very parameter-efficient

Worked Example

```
BERT contextual embeddings for 'bank':

Sentence A: 'I deposited money at the bank.'
BERT vector for 'bank' ~ financial institution cluster

Sentence B: 'We fished on the river bank.'
BERT vector for 'bank' ~ geographical feature cluster

cosine_sim(bank_A, bank_B) = 0.31 (different contexts -> different vectors!)
Static Word2Vec: same vector for both -> cosine_sim = 1.0
```

6. Attention Mechanisms and Transformers

The **Transformer** (Vaswani et al., 2017, 'Attention Is All You Need') is arguably the most important architecture in modern deep learning. It replaced RNNs for sequence modelling by relying entirely on **attention mechanisms** — eliminating recurrence and enabling massive parallelisation. It is the foundation of BERT, GPT, T5, LLaMA, and virtually all large language models.

6.1 The Attention Problem with RNNs

- Seq2Seq bottleneck: entire input compressed to one fixed vector — loses information for long sequences.
- Bahdanau et al. (2015) introduced additive attention: at each decoder step, look at ALL encoder states and compute a weighted sum.
- Intuition: when translating word 3, attend strongly to relevant encoder positions, not just the final state.

```
Context vector: c_t = sum_i alpha_{t,i} * h_i [weighted sum of encoder states]
Attention weight: alpha_{t,i} = softmax(e_{t,i}) where e_{t,i} = score(s_{t-1}, h_i)
Bahdanau score: e_{t,i} = v^T * tanh(W_a*s_{t-1} + U_a*h_i) [additive/concat]
Luong score: e_{t,i} = s_t^T * W_a * h_i [multiplicative/dot]
```

6.2 Scaled Dot-Product Attention

The Transformer uses a specific, parallelisable form of attention based on three projections: **Query (Q)**, **Key (K)**, and **Value (V)** matrices. Conceptually: the query is 'what am I looking for?', keys are 'what do I contain?', values are 'what information do I carry?'.

```
Attention(Q, K, V) = softmax( Q*K^T / sqrt(d_k) ) * V
Q = X * W^Q, K = X * W^K, V = X * W^V [learned linear projections]
d_k = dimension of keys (typically 64); sqrt(d_k) prevents vanishing gradients in softmax
```

Worked Example

```
Scaled Dot-Product Attention step by step:

Input sequence: ['The', 'cat', 'sat'] -> each embedded to d_model=4 vectors
X = [[1.0, 0.2, 0.5, 0.3], <- 'The'
[0.3, 1.0, 0.8, 0.4], <- 'cat'
[0.5, 0.6, 1.0, 0.2]] <- 'sat'

Q, K, V computed by linear projection (W^Q, W^K, W^V are 4x4 matrices)

Scores = Q * K^T / sqrt(4) -> 3x3 score matrix
Score[1,2] = Q['cat'] . K['sat'] / 2 = similarity of cat to sat
```

```
After softmax: attention weights sum to 1 per row
e.g. Row for 'cat': [0.1, 0.7, 0.2] -> attends 70% to itself

Output = attention_weights * V -> 3x4 output (same shape as input)
```

6.3 Multi-Head Attention

Rather than computing attention once, the Transformer uses **h parallel attention heads** (h=8 in the original paper), each with different learned projections (d_k = d_model/h = 64). Different heads can attend to different types of relationships simultaneously — one head might capture syntactic structure, another semantic similarity, another coreference.

```
head_i = Attention(Q*W_i^Q, K*W_i^K, V*W_i^V)
MultiHead(Q,K,V) = Concat(head_1, ..., head_h) * W^O
Original paper: h=8, d_model=512, d_k=d_v=64 -> each head operates on 64-dim subspace
Total attention params per layer: 4 * d_model^2 (for W^Q, W^K, W^V, W^O)
```

6.4 Positional Encoding

Attention is **permutation-invariant** — it treats the sequence as an unordered set. Positional encodings are added to the embeddings to inject sequence order information. The original Transformer uses sinusoidal encodings:

```
PE(pos, 2i) = sin(pos / 10000^(2i / d_model))
PE(pos, 2i+1) = cos(pos / 10000^(2i / d_model))
Each position gets a unique d_model-dimensional vector; close positions are similar
```

- Sinusoidal: no learned parameters; extrapolates to unseen sequence lengths.
- Learned positional embeddings (BERT, GPT): trainable vectors for each position; typically performs similarly.
- Rotary Position Embedding (RoPE, LLaMA): encodes relative positions via rotation in embedding space.
- ALiBi (attention with linear biases): adds position bias to attention scores — works well for long contexts.

6.5 Transformer Block

A Transformer block consists of two sub-layers, each wrapped with a residual connection and layer norm:

```
Sub-layer 1: Multi-Head Self-Attention
Sub-layer 2: Position-wise Feed-Forward Network (FFN)
Output = LayerNorm(x + SubLayer(x)) [Add & Norm / Pre-Norm or Post-Norm]
```

Feed-Forward Sub-layer

```
FFN(x) = max(0, x*W_1 + b_1) * W_2 + b_2 [two linear + ReLU; often GELU]
d_model=512 -> d_ff=2048 (4x expansion), then back to 512
```

The FFN is applied independently to each position — it is a **per-token** MLP that stores factual knowledge in its weights. Research shows FFN layers act as key-value memory stores.

6.6 The Full Transformer Architecture

Encoder (BERT-style)

- N=6 identical encoder blocks (N=12 for BERT-base, N=24 for BERT-large).
- Input: token embeddings + positional encodings.
- Each block: Multi-Head Self-Attention + FFN, both with Add&Norm.;
- Self-attention: each token attends to ALL other tokens (fully bidirectional).
- Output: rich contextual representations for each input token.

Decoder (GPT-style)

- N=6 identical decoder blocks.
- Additional **masked** self-attention: tokens can only attend to PREVIOUS positions (causal).
- In encoder-decoder models (T5, original Transformer): cross-attention layer connects decoder to encoder output.
- Autoregressive generation: produce one token at a time, feeding back as input.

Encoder-Decoder (T5, Original Transformer)

```
Encoder reads full input; Decoder attends to encoder via cross-attention
Cross-Attention: Q from decoder, K and V from encoder output
```

6.7 Transformer Training: Masking & Objectives

Objective	Model	Description
Masked Language Model (MLM)	BERT	Mask 15% of tokens; predict masked tokens using bidirectional context
Next Sentence Prediction (NSP)	BERT	Predict if sentence B follows sentence A
Causal Language Model (CLM)	GPT	Predict next token; trained autoregressively
Span Denoising	T5	Mask spans; decoder reconstructs them
Contrastive (SimCSE)	SimCSE	Push similar sentence pairs together in embedding space

6.8 Complexity and Scaling

```
Self-attention complexity: O(n^2 * d) [n = sequence length, d = model dim]
FFN complexity: O(n * d * d_ff)
Memory: attention matrix is n x n per head -> bottleneck for long sequences
```

- **Efficient Attention variants** to handle long contexts:
- Sparse Attention (Longformer): local window + global tokens -> O(n * w) where w = window size.
- Linformer: project K and V to low rank -> O(n) complexity.
- Flash Attention: IO-aware exact attention; no approximation; 2-4x faster via GPU memory tiling.
- Grouped Query Attention (GQA, LLaMA 2/3): fewer key-value heads -> reduces KV cache in inference.

Scaling Laws (Kaplan et al., 2020)

```
L(N) ~ N^(-0.076) [loss scales as power law with parameters N]
L(C) ~ C^(-0.050) [loss scales with compute C]
L(D) ~ D^(-0.095) [loss scales with dataset size D]
```

6.9 Major Transformer Models Timeline

Year	Model	Key Innovation	Params
2017	Transformer	Original encoder-decoder; attention is all you need	65M
2018	GPT-1	Unidirectional transformer decoder; GPT pretraining	117M
2018	BERT-base/large	Bidirectional; MLM+NSP; fine-tuning paradigm	110M/340M
2019	GPT-2	Larger GPT; zero-shot; 'too dangerous to release'	1.5B
2019	T5	Unified text-to-text framework; all tasks as seq2seq	11B
2020	GPT-3	Few-shot in-context learning; emergent capabilities	175B
2021	CLIP	Contrastive image-text; zero-shot vision	400M
2022	ChatGPT	RLHF fine-tuning for instruction following	~175B
2023	LLaMA-2	Open weights; GQA; long context; RLHF	7-70B
2024	LLaMA-3	Improved tokenizer (128k vocab); multilingual	8-405B

6.10 Self-Attention: Types Summary

Type	What Q, K, V come from	Used In	Property
Self-Attention	All from same sequence X	BERT encoder, GPT decoder	Each token attends to all others in same seq
Masked Self-Attn	Same seq; future masked	GPT, decoder blocks	Causal: token only sees past
Cross-Attention	Q from decoder, K/V from encoder	T5, orig. Transformer	Decoder queries encoder representations
Global Attention	Some tokens attend to all	Longformer	Efficient for long documents
Multi-Query Attn	Shared K/V across heads	PaLM, Falcon	Faster inference; smaller KV cache

The Paradigm Shift: From RNNs to Transformers

RNNs: sequential computation (O(n) steps); hard to parallelise; vanishing gradients over long range.

Transformers: parallel computation (O(1) sequential steps); any two tokens can directly attend; no recurrence. This made training on massive datasets feasible with modern GPU/TPU hardware.

The Transformer enabled scaling from millions to billions to trillions of parameters, unlocking emergent capabilities — in-context learning, chain-of-thought reasoning, code generation — that were impossible with previous architectures.